Python DB and Framework
Theory Assignment

Q1. HTML in Django – Introduction

HTML is the basic language used to create web pages and structure content. In Django, HTML is used with templates to generate dynamic pages. Templates allow us to insert Python data into HTML using variables and tags. The view function sends data to templates using a context dictionary. This helps in separating backend logic and frontend design. It makes the code clean and easy to maintain. Template inheritance allows reuse of common layouts. It reduces duplication and improves development speed. Django Template Language (DTL) is simple and powerful. It supports loops, conditions, and filters. Overall, HTML with Django templates helps in building dynamic and user-friendly websites.
Example:
render(request,'home.html',{'name':'Anshu'})

Q2. CSS in Django – Integration

CSS is used to style web pages including layout, colors, fonts, and responsiveness. In Django, CSS is managed as static files. We use the static folder to store CSS files. To connect CSS in templates, we use {% load static %}. This keeps design separate from logic. It helps maintain consistent design across the application. Bootstrap can also be used to quickly create responsive designs. CSS improves user experience and UI. It also helps in branding and visual appeal. Proper CSS management is important for large projects. Overall, CSS plays a key role in frontend development in Django.
Example:
<link rel='stylesheet' href='{% static "style.css" %}'>

Q3. JavaScript in Django

JavaScript is used to add interactivity and dynamic behavior to web pages. It runs on the client side in the browser. In Django, JavaScript can be added inside templates or as static files. It handles events like clicks, form validation, and animations. JavaScript can also communicate with Django views using AJAX or Fetch API. This allows data exchange without page reload. It improves user experience and performance. JavaScript is widely used for real-time updates. It also helps in building modern web applications. Combining JavaScript with Django makes applications more interactive.
Example:
alert('Form Submitted')

Q4. Django Framework – Overview

Django is a high-level Python web framework used to build web applications quickly. It follows the DRY principle to reduce repetition. Django provides built-in features like ORM, templates, authentication, and admin panel. It is secure and scalable. It protects against common attacks like SQL injection and CSRF. Django supports multiple databases. It is widely used in real-world applications. The framework follows MVT architecture. It helps developers focus on logic rather than configuration. Overall, Django simplifies web development.
Example:
django-admin startproject projectname

Q5. Virtual Environment

A virtual environment is used to create an isolated Python environment for each project. It helps manage dependencies separately. Different projects can use different versions of libraries. This avoids conflicts between packages. It improves project portability and stability. Python provides venv module for creating virtual environments. After activation, packages are installed locally. It is widely used in development and deployment. It also helps in team collaboration. Overall, virtual environments are essential for Python projects.

Example:
python -m venv myenv

Q6. Project and App in Django

In Django, a project is the main application that contains settings and configurations. An app is a module inside the project. Each app handles specific functionality like users or products. Multiple apps can exist in one project. This makes the structure modular and reusable. Apps contain models, views, templates, and urls. It improves organization of code. It also helps in scaling projects. Developers can reuse apps in other projects. Overall, project and app structure is very powerful in Django.
Example:
python manage.py startapp appname

Q7. MVT Architecture

Django follows the Model-View-Template architecture. Model handles database operations. View contains business logic. Template is used for displaying data to the user. Django itself acts as controller. It separates concerns clearly. This improves maintainability and readability. Request goes from URL to view, then to template. Data flows through models. It helps in building scalable applications. MVT is similar to MVC pattern. Overall, it makes development structured and clean.
Example:
Model.objects.all()

Q8. Django Admin Panel

Django provides a built-in admin panel for managing data. It allows CRUD operations without writing extra code. Models are registered in admin.py. Superuser is created to access admin panel. It is accessible at /admin URL. It saves development time. Admin panel can be customized using ModelAdmin. We can add filters, search, and display fields. It is very useful for backend management. Overall, admin panel is a powerful feature of Django.

Example:
admin.site.register(Model)

Q9. URL Routing

URL routing connects URLs to view functions in Django. It is defined in urls.py file. The path() function is used for simple routing. It maps URL patterns to views. Named URLs improve flexibility and maintainability. Templates use {% url %} tag to generate links. It helps in navigation between pages. It also supports dynamic URLs using parameters. Overall, URL routing is essential for web applications.
Example:
path('home/',views.home,name='home')

Q10. Form Validation

Form validation ensures that user input is correct and secure. JavaScript is used for client-side validation. It checks fields before sending data to server. It improves user experience. Django also provides server-side validation. Both validations are important for security. Common checks include email format and phone length. Validation prevents invalid data entry. It reduces server load. Overall, form validation is essential in web applications.
Example:
if(phone.length!=10){alert('Invalid')}

Q11. Database Connectivity

Django supports databases like SQLite and MySQL. SQLite is default database for development. Database settings are defined in settings.py. ORM connects models to database tables. Migrations are used to update database schema. makemigrations creates migration files. migrate applies changes. It helps in managing data easily. Django abstracts SQL queries. Overall, database connectivity is simple and powerful.
Example:
python manage.py migrate

Q12. ORM and QuerySet

ORM stands for Object Relational Mapper. It allows database operations using Python code. It removes the need for writing SQL queries. QuerySet is a collection of database objects. Methods like filter(), get(), create() are used. It is database independent. It improves readability and security. It prevents SQL injection. Overall, ORM simplifies database handling.
Example:
Model.objects.filter(name='Anshu')

Q13. Django Forms and Authentication

Django forms are used to collect and validate user input. ModelForm creates forms from models automatically. Authentication system handles login and logout. User model is built-in with secure password hashing. Sessions are used to manage login state. login_required decorator protects views. It improves security of application. It also supports password reset features. Overall, forms and authentication are important for user management.
Example:
authenticate(request,username,password)

Q14. AJAX CRUD Operations

AJAX allows asynchronous communication between client and server. It updates data without page reload. Used for Create, Read, Update, Delete operations. Fetch API or jQuery is used to send requests. Django returns JSON response. It improves speed and user experience. It is used in dynamic applications. CSRF token is required for security. Overall, AJAX makes applications more interactive.
Example:
fetch('/delete/1/')

Q15. Customizing Admin Panel

Django admin panel can be customized using ModelAdmin class. We can define list_display to show fields. search_fields allows searching records. list_filter adds filtering options. It improves usability of admin panel. We can also add custom actions. It helps in managing data efficiently. Overall, customization makes admin more powerful.
Example:
list_display=['name']

Q16. Paytm Integration

Paytm is a popular payment gateway in India. It allows online transactions securely. Merchant ID and key are required for integration. Checksum is generated for security. User is redirected to Paytm page for payment. After payment, response is verified. Transaction details are stored in database. It supports multiple payment options. Overall, it is useful for e-commerce applications.
Example:
generateSignature()

Q17. GitHub Deployment

GitHub is used for version control and code hosting. It helps track changes in code. Developers push code to repository. Branches are used for feature development. Pull requests are used for merging code. Sensitive data should not be uploaded. It supports collaboration in teams. CI/CD can be integrated. Overall, GitHub is essential for modern development.
Example:
git push origin main

Q18. PythonAnywhere Deployment

PythonAnywhere is a cloud platform for hosting Python applications. It supports Django deployment easily. Users can upload project using Git. Virtual environment is created for dependencies. WSGI file is configured for app. Static files are collected. App runs online with URL. Logs help in debugging. Overall, it is beginner-friendly hosting platform.

Example:
username.pythonanywhere.com

Q19. Social Authentication

Social authentication allows login using Google or Facebook accounts. It uses OAuth2 protocol. It reduces signup effort for users. Django-allauth package is used. Client ID and secret are required. User is redirected to provider login page. After login, data is returned to app. It improves user experience. Overall, it is secure and convenient login method.
Example:
Login with Google

Q20. Google Maps API

Google Maps API is used to display maps in web applications. It requires API key from Google Cloud. Maps are added using JavaScript. Markers are used to show locations. It is useful for location-based services. It improves user experience. It can show routes and directions. Used in apps like doctor finder. Overall, it adds powerful map features.
Example:
new google.maps.Map()